

Shell Commands Reference

A Complete Beginner's Guide

Linux / WSL · PowerShell · Command Prompt
Tailored for the OnDevice HAR FQT+RigL Master Thesis Project

This guide explains every terminal command you will use when building, training, and evaluating the FQT+RigL model on PC (Windows / WSL). Each command is explained from scratch — no prior Linux or terminal experience is assumed. Side-by-side comparisons show the same action in all three environments so you always know which version to type.

Contents

#	Section	Page
1	What Is a Terminal (Shell)?	2
2	The Three Shells: Linux/WSL, PowerShell, CMD	2
3	How to Open Each Shell	3
4	Reading the Prompt — How to Know Where You Are	3
5	Understanding File Paths	4
6	Navigation Commands — Moving Between Folders	5
7	File and Folder Operations	7
8	WSL — Running Linux Inside Windows	9
9	Installing Software in WSL (apt)	11
10	CMake — Configuring and Building C Projects	12
11	Complete Project Workflow — Step by Step	14
12	Common Errors and How to Fix Them	16
13	Quick Reference Card	18

1. What Is a Terminal (Shell)?

When you use a computer normally, you click icons and menus — that is called a **Graphical User Interface (GUI)**. A **terminal** (also called a **shell** or **command line**) is a text-only window where you type commands to make the computer do things.

Think of it like giving spoken instructions to a very precise assistant. Instead of clicking File → Open, you type the instruction. The advantage is speed: tasks that take many clicks become one short command. For software development — especially building C projects — the terminal is essential because compilation tools like **cmake** and **gcc** have no GUI.

TIP: You do not need to memorise everything at once. Bookmark this PDF and look up commands as you need them.

Basic rules that apply to ALL shells

Rule	Explanation
Press Enter to run a command	Type your command, then press Enter. Nothing happens until you press Enter.
Commands are case-sensitive in Linux	Is and LS are different in Linux/WSL. PowerShell and CMD are not case-sensitive.
Spaces matter in paths	If a folder name has spaces, wrap the whole path in quotes: <code>cd "My Folder"</code>
Press Ctrl+C to stop	If a program is running and you want to stop it, press Ctrl+C.
Up arrow = previous command	Press the Up arrow key to recall the last command you ran. Press it again for the one before that.
Tab = autocomplete	Type the first few letters of a file/folder name and press Tab. The shell will complete it for you.
# starts a comment	Any line starting with # is ignored. Comments are notes for humans.

2. The Three Shells: Linux/WSL, PowerShell, and CMD

Windows gives you multiple shell environments. This project uses three. Understanding which one you are in — and which to use — prevents most beginner errors.

Shell	Full Name	Background Colour	When to Use It
Linux / WSL (bash)	Windows Subsystem for Linux — Bash shell	Usually dark/black with coloured prompt	Building the C project with cmake and gcc. Installing Linux packages. Running Python scripts. ALL compilation work goes here.
PowerShell	Windows PowerShell or PowerShell 7	Blue background (classic) or dark	Managing files and folders on Windows. Renaming CMakeLists files. General Windows automation.
Command Prompt (CMD)	cmd.exe — the classic Windows shell	Black background no colour	Simple Windows commands. Shutting down WSL (wsl --shutdown). Legacy batch scripts.

TIP: For this project, you will spend most of your time in **Linux / WSL**. Use PowerShell only for Windows file management tasks (renaming files, etc.).

WARNING: Commands are NOT interchangeable between shells. A Linux command like rm will not work in CMD. A PowerShell command like Remove-Item will not work in Linux. Always check which shell you are in before typing.

3. How to Open Each Shell

Opening Linux / WSL

WSL must be installed first (it comes pre-installed on Windows 11, and can be installed on Windows 10 via the Microsoft Store).

How to open WSL

Method 1: Press the Windows key, type 'wsl', press Enter.
Method 2: Open PowerShell or CMD and type: wsl
Method 3: Right-click the Start button → 'Terminal' → open a new tab and choose Ubuntu.

Opening PowerShell

How to open PowerShell

Method 1: Press Windows key, type 'PowerShell', press Enter.
Method 2: Press Win+X (right-click Start button) → 'Windows PowerShell' or 'Windows Terminal'.
Method 3: Inside any open terminal window, click the + button (new tab) and select PowerShell.

Opening Command Prompt (CMD)

How to open Command Prompt

Method 1: Press Windows key, type 'cmd', press Enter.
Method 2: Press Win+R, type 'cmd', press Enter.
Method 3: In the search bar type 'Command Prompt'.

TIP: Windows Terminal (available from the Microsoft Store) is the modern replacement that lets you have Linux, PowerShell, and CMD all as tabs in one window. Highly recommended.

4. Reading the Prompt — How to Know Where You Are

The **prompt** is the text that appears before the flashing cursor. It tells you which shell you are in and where on the filesystem you currently are. Reading the prompt correctly prevents you from running commands in the wrong place.

Linux / WSL prompt — examples

```
matef@LAPTOP:~$ <-- Linux / WSL
| | |
| | +-- $ means normal user (# means administrator/root)
| +----- name of the computer
+----- your Linux username

matef@LAPTOP:~/har_build$ <-- WSL, currently inside ~/har_build
matef@LAPTOP:/mnt/f/project$ <-- WSL, on Windows drive F:
```

PowerShell prompt — examples

```
PS C:\Users\matef> <-- PowerShell
| |
| +-- current folder on Windows (C:\Users\matef)
+----- 'PS' always appears at the start in PowerShell

PS C:\Users\matef\har_project> <-- inside a subfolder
```

Command Prompt prompt — examples

```
C:\Users\matef> <-- Command Prompt
|
+-- current folder (no prefix, just the path and >)

F:\technical material> <-- CMD on drive F:
```

TIP: If you are ever unsure which shell you are in, look at the very beginning of the line. PS = PowerShell, username@computer = WSL/Linux, just a path = CMD.

5. Understanding File Paths

A **path** is the address of a file or folder on the computer. Think of it like a postal address — it tells the computer exactly where something is located. Linux and Windows use different formats.

5.1 Windows paths (PowerShell and CMD)

Windows uses a **drive letter** (like C: or F:) followed by backslashes (\) between folder names.

Windows path format

```
F:\technical material\Master thesis\claudio\on_device training
^ ^ ^
| | +-- innermost folder
| +-- subfolder
+-- drive letter (F:)
```

5.2 Linux paths (WSL / bash)

Linux has no drive letters. Everything starts from / (called 'root'). Linux uses forward slashes (/) between folder names.

Linux path format

```
/home/matef/har_build
^ ^ ^
| | +-- your project folder
| +-- your Linux home folder
+-- root of the entire filesystem

~ means 'my home folder' – a shortcut for /home/matef
```

5.3 How WSL sees Windows drives

WSL mounts Windows drives under **/mnt/**. Drive F:\ becomes /mnt/f/, drive C:\ becomes /mnt/c/.

Windows ↔ WSL path translation

```
Windows path: F:\technical material\Master thesis\...
WSL path: /mnt/f/technical material/Master thesis/...

Windows path: C:\Users\matef\Documents
WSL path: /mnt/c/Users/matef/Documents

Rule: replace F:\ with /mnt/f/
      replace \ with /
```

5.4 Absolute vs. relative paths

Type	Starts With	Meaning	Example
Absolute path	/ or C:\ or F:\	Full address from the very top of the filesystem. Always works no matter where you currently are.	/mnt/f/project/src/main.c
Relative path	A folder name or . or ..	Address relative to where you currently are. . means 'here'. .. means 'go up one level'.	src/main.c (if you are already in /mnt/f/project)

TIP: When in doubt, use absolute paths — they are unambiguous. Relative paths are shorter but only work if you are in the right folder.

6. Navigation Commands — Moving Between Folders

The most common task in a terminal is navigating between folders (also called directories). These four commands cover 90% of navigation.

6.1 pwd — Print Working Directory (where am I?)

pwd stands for 'print working directory'. It prints the full path of the folder you are currently in. Use it whenever you are unsure where you are.

Linux / WSL	PowerShell	Command Prompt
<pre>\$ pwd /home/matef/har_build</pre> In WSL: pwd shows the Linux path	<pre>PS> pwd Path ---- C:\Users\matef</pre> PowerShell: pwd or Get-Location	<pre>C:\> cd C:\Users\matef</pre> CMD: typing cd with no argument shows current folder

6.2 cd — Change Directory (move to a different folder)

cd stands for 'change directory'. It moves you to a different folder. Think of it as double-clicking a folder icon, but in text form.

cd examples (Linux / WSL)

```
Syntax: cd

# Go into a subfolder called 'src' (relative path)
cd src

# Go into an absolute path
cd "/mnt/f/technical material/Master thesis/claude/on_device training"

# Go up one level (to the parent folder)
cd ..

# Go to your home folder
cd ~

# Go to the root of the filesystem
cd /
```

cd examples (PowerShell / CMD)

```
# Important: wrap paths with spaces in DOUBLE QUOTES
cd "F:\technical material\Master thesis"

# Go up one level
cd ..

# Go to your user home
cd $HOME # PowerShell
cd %USERPROFILE% # CMD
```

WARNING: In Linux, folder names are case-sensitive. 'cd Documents' and 'cd documents' go to different places (or one may not exist). In Windows, case does not matter.

6.3 ls — List files and folders

ls (Linux) or dir (Windows CMD) shows everything inside the current folder. Use it to see what is here before navigating further.

Linux / WSL	PowerShell	Command Prompt
<pre>\$ ls # simple list \$ ls -la # detailed list # (shows hidden files, # sizes, permissions) -l = long format, -a = all (including hidden files starting with .)</pre>	<pre>PS> ls # works in PowerShell too PS> dir # alias – same result PS> ls -la # NOT valid in PS PS> Get-ChildItem # full command PowerShell's ls is an alias for Get-ChildItem</pre>	<pre>C:\> dir # shows all files C:\> dir /a # include hidden dir is the only option in CMD</pre>

TIP: Run ls (or dir) whenever you are not sure what is in the current folder. It is the equivalent of opening a folder window in Explorer.

6.4 Navigating to the project folder — full example

This is the exact sequence to get from anywhere to the project folder inside WSL, then build it:

Complete navigation example

```
# Step 1: Open WSL (type 'wsl' in PowerShell or search 'WSL')
# You will see the prompt: matef@LAPTOP:~$

# Step 2: Navigate to the project
cd "/mnt/f/technical material/Master thesis/claude/on_device training"

# Step 3: Confirm you are in the right place
pwd
# Expected output:
# /mnt/f/technical material/Master thesis/claude/on_device training

# Step 4: List what is here
ls
# You should see: OnDeviceTraining OnDeviceTraining_HAR_FQT_RigL ...
```

7. File and Folder Operations

These commands let you create, delete, rename, copy, and inspect files and folders without using the mouse. They work across all three shells, but the command names differ.

7.1 mkdir — Make a new folder

mkdir creates a new empty folder. It stands for 'make directory'.

Linux / WSL	PowerShell	Command Prompt
<pre>\$ mkdir build \$ mkdir -p a/b/c # -p creates parent folders # that don't exist yet -p prevents error if folder already exists</pre>	<pre>PS> mkdir build PS> mkdir -Force a\b\c # -Force creates parents Both mkdir and md work in PowerShell</pre>	<pre>C:\> mkdir build C:\> md build # md is an alias md is shorthand for mkdir in CMD</pre>

7.2 rm / del — Delete files or folders

Deleting from the terminal is **permanent** — files do NOT go to the Recycle Bin. Double-check before running these commands.

WARNING: There is NO undo in the terminal. Once you delete with `rm -rf` or `rmdir /s /q`, the files are gone. Be very careful with these commands.

Linux / WSL	PowerShell	Command Prompt
<pre>\$ rm file.txt # delete one file \$ rm -rf foldername # delete a folder and # EVERYTHING inside it # -r = recursive (subfolders) # -f = force (no confirmation)</pre>	<pre>PS> Remove-Item file.txt # delete one file PS> Remove-Item -Recurse -Force foldername # delete folder + contents # -Recurse = include subfolders # -Force = no confirmation</pre>	<pre>C:\> del file.txt # delete one file C:\> rmdir /s /q foldername # delete folder + contents # /s = include subfolders # /q = quiet (no confirmation)</pre>

7.3 mv / Rename-Item — Rename or move a file

In Linux, **mv** (move) handles both renaming and moving. In Windows, renaming uses **Rename-Item** (PowerShell) or **ren** (CMD).

Linux / WSL	PowerShell	Command Prompt
<pre>\$ mv old.txt new.txt # rename old.txt to new.txt \$ mv file.txt /mnt/f/other/ # move file to a different folder</pre>	<pre>PS> Rename-Item old.txt new.txt # rename a file PS> Move-Item file.txt C:\other\ # move a file</pre>	<pre>C:\> ren old.txt new.txt # rename a file C:\> move file.txt C:\other\ # move a file</pre>

Renaming the CMake files (which you had to do in this project) looks like this:

Renaming CMakeLists files — project-specific example

```
# In PowerShell – rename the Pico cmake file first, then rename the PC file:

PS C:\...\OnDeviceTraining_HAR_FQT_RigL> Rename-Item CMakeLists.txt CMakeLists_Pico.txt
PS C:\...\OnDeviceTraining_HAR_FQT_RigL> Rename-Item CMakeLists_PC.txt CMakeLists.txt

# Why rename in this order?
# If CMakeLists.txt already exists, you cannot rename CMakeLists_PC.txt
# to that name until the existing one has been moved out of the way first.
```

7.4 cp / Copy-Item — Copy a file

Linux / WSL	PowerShell	Command Prompt
\$ cp source.txt dest.txt # copy one file \$ cp -r folder/ dest/ # copy entire folder (-r)	PS> Copy-Item source.txt dest.txt PS> Copy-Item -Recurse folder dest	C:\> copy source.txt dest.txt C:\> xcopy folder dest /E /I # /E includes subfolders

7.5 cat / type — View file contents without opening an editor

Linux / WSL	PowerShell	Command Prompt
\$ cat filename.txt # print file to screen \$ head -20 filename.txt # show first 20 lines only	PS> Get-Content filename.txt # print file to screen PS> Get-Content file.txt Select -First 20 # first 20 lines	C:\> type filename.txt # print file to screen

7.6 Chaining commands — running multiple commands in sequence

You often want to run several commands in a row. The way to chain them differs between shells — this is a common source of errors.

Symbol	Shell	Meaning	Example
&&	Linux / CMD	Run the second command ONLY if the first succeeded (exit code 0 = success).	mkdir build && cd build
;	Linux / PowerShell	Run both commands regardless of whether the first succeeded or failed.	mkdir build ; cd build
&	CMD only	Run both commands regardless of result (CMD version of ;).	mkdir build & cd build
	All shells	Pipe — send the OUTPUT of the first command as INPUT to the second. Used for filtering.	ls grep main

WARNING: && does NOT work in PowerShell. PowerShell treats && as a bitwise AND operator and shows a syntax error. Use ; or separate lines instead.

The && vs ; mistake — very common for beginners

WRONG in PowerShell – causes error:

```
mkdir build_pc && cd build_pc
```

CORRECT in PowerShell (two options):

```
mkdir build_pc
```

```
cd build_pc
```

OR on one line:

```
mkdir build_pc ; cd build_pc
```

8. WSL — Running Linux Inside Windows

WSL (Windows Subsystem for Linux) is a feature of Windows 10/11 that runs a real Linux environment (Ubuntu) directly inside Windows, without needing a separate computer or virtual machine.

For this project, WSL is essential because the compiler (GCC 10+) and build tools required by the OnDeviceTraining framework are only available in Linux. The MinGW GCC bundled with CodeBlocks on Windows is version 8.1 — too old for the framework's C11 patterns.

8.1 WSL control commands — typed in PowerShell or CMD

Command	Where to Type It	What It Does
wsl	PowerShell or CMD	Open a WSL bash terminal. You will see the Linux prompt.
wsl --list	PowerShell or CMD	Show all installed Linux distributions (e.g. Ubuntu-22.04).
wsl --version	PowerShell or CMD	Show the WSL and kernel version numbers.
wsl --shutdown	PowerShell or CMD	Stop ALL running WSL instances immediately. This releases any file locks WSL has on Windows folders, which is necessary when Windows cannot delete a folder that WSL has open.
exit	Inside WSL	Leave the WSL terminal and return to PowerShell or CMD.

8.2 How WSL and Windows share files

WSL and Windows run side by side and can see each other's files, but they use different path formats to refer to the same locations.

Windows ↔ WSL path translation table

```
From Windows (PowerShell/CMD) From WSL (bash)
```

```
F:\technical material\project /mnt/f/technical material/project  
C:\Users\matef\Documents /mnt/c/Users/matef/Documents  
C:\Users\matef /mnt/c/Users/matef OR ~
```

Translation rule:

Drive letter + colon → /mnt/ + lowercase drive letter

Backslash \ → Forward slash /

8.3 Why we build in the WSL native home folder

You might expect to build the project directly in the Windows folder (F:\...) from inside WSL. This usually works, but some file operations (creating and deleting folders, permissions) behave unexpectedly on Windows-mounted drives accessed from Linux. This is called a **DrvFS limitation**.

The solution is to build in the Linux native home folder (`~/har_build`), which has no such restrictions. The source code stays in `F:\`, but the compiled output goes into `~/har_build`.

Source vs. build directory — the split

Source code (stays in Windows): /mnt/f/.../OnDeviceTraining_HAR_FQT_RigL/
Build output (goes into Linux): ~/har_build/
(which is /home/matef/har_build/)

cmake is told where the source is:
cmake "/mnt/f/.../OnDeviceTraining_HAR_FQT_RigL"

cmake reads CMakeLists.txt from that path and writes build files
to the CURRENT FOLDER (~/har_build).

TIP: ~/har_build is persistent — it survives closing and reopening WSL. You only need to run cmake once (to configure). After that, just run cmake --build . to recompile after code changes.

8.4 When WSL locks a Windows folder — and how to fix it

WSL sometimes holds a file lock on a Windows folder, preventing you from deleting it from Windows Explorer or PowerShell. The fix is always the same:

Fixing 'The process cannot access the file' error

Step 1: In PowerShell or CMD, shut down WSL:
wsl --shutdown

Step 2: Wait 5 seconds, then delete the folder from Windows:
Remove-Item -Recurse -Force build_pc
or in CMD:
rmdir /s /q build_pc

Step 3: Reopen WSL:
wsl

9. Installing Software in WSL (apt)

WSL runs Ubuntu Linux. Ubuntu uses a package manager called **apt** (Advanced Package Tool) to install software. Think of it like the App Store, but operated entirely from the terminal.

sudo means 'superuser do' — it temporarily grants you administrator privileges for one command. Linux requires this for installing software. WSL will ask for your Linux password the first time.

9.1 Installing all required tools for this project

Run these commands **one at a time**. Wait for each one to finish before typing the next.

Full installation sequence

```
# Command 1: Update the list of available packages
# (This does NOT install anything – it just refreshes the catalogue)
sudo apt update

# What you will see:
# [sudo] password for matef: <- type your Linux password, press Enter
# Hit:1 http://archive.ubuntu.com ... <- downloading package lists
# Reading package lists... Done

# Command 2: Install cmake, gcc, g++, and make
# -y means 'yes to all questions' (so it doesn't ask you to confirm each package)
sudo apt install -y cmake gcc g++ make

# What you will see:
# Reading package lists... Done
# Building dependency tree... Done
# The following NEW packages will be installed: ...
# Do you want to continue? [Y/n] <- press Enter (or -y skips this)
# ... lots of download/install output ...
# Processing triggers for ... Done

# Command 3: Verify everything installed correctly
cmake --version # should print: cmake version 3.x.x
gcc --version # should print: gcc (Ubuntu ...) 12.x.x
make --version # should print: GNU Make 4.x
```

9.2 Common apt commands reference

Command	What It Does
sudo apt update	Refresh the package list (run this first, always)
sudo apt install -y package	Install a package. -y skips confirmation.
sudo apt remove package	Uninstall a package.
sudo apt upgrade	Update all installed packages to their latest versions.
apt search keyword	Search for a package by name or keyword.
dpkg -l grep name	Check if a package is already installed.

TIP: If apt install fails with 'package not found', run `sudo apt update` first and try again. The package list may be outdated.

WARNING: Do NOT install multiple packages across separate commands in a rapid sequence without waiting. Paste one command, wait for the Done message, then paste the next.

10. CMake — Configuring and Building C Projects

CMake is a build system generator. It reads a file called **CMakeLists.txt** and produces the actual build instructions that make/ninja then use to compile your C code.

Building a project always happens in **two separate steps**: **configure** (cmake reads CMakeLists.txt and prepares everything), then **build** (cmake --build . actually compiles the code).

10.1 The CMake two-step workflow

The three-step cmake workflow

STEP 1: Create a separate build folder and go into it
(NEVER run cmake inside your source code folder)

```
mkdir -p ~/har_build  
cd ~/har_build
```

STEP 2: Configure – tell cmake where the source code is
cmake "/mnt/f/technical material/Master thesis/claude/on_device
training/OnDeviceTraining_HAR_FQT_RigL"

cmake reads CMakeLists.txt from that source folder
and writes build files into ~/har_build (your current folder).

Successful output ends with:

```
-- Build files have been written to: /home/matef/har_build
```

STEP 3: Build – compile the code

```
cmake --build .
```

. means 'this folder' (~/har_build, where cmake wrote the build files)

Successful output ends with:

```
[100%] Built target har_fqt_rigl
```

10.2 What each cmake output percentage means

Output line	What is happening
-- Found Python...	CMake is checking what tools are available on your system
-- Configuring done	cmake has read all CMakeLists.txt files successfully
-- Build files have been written to: ...	Configure is finished; you can now run cmake --build
[10%] Compiling C object ...	GCC is compiling one .c source file
[80%] Linking C executable ...	All .o object files are being linked into the final program
[100%] Built target har_fqt_rigl	Compilation succeeded. The executable is ready.

10.3 Important: CMakeLists.txt must be named exactly that

CMake always looks for a file called exactly CMakeLists.txt (capital C, capital L, .txt extension). There is **no** -f flag to specify a different filename.

The -f flag mistake — does not exist in cmake

```
# WRONG – this flag does not exist:
cmake -f CMakeLists_PC.txt .. # ERROR: Unknown argument '-f'

# CORRECT – rename the file first, then run cmake:
# (In PowerShell)
Rename-Item CMakeLists.txt CMakeLists_Pico.txt
Rename-Item CMakeLists_PC.txt CMakeLists.txt
# Now cmake will find it automatically
```

10.4 Re-building after code changes

After you edit a .c or .h file, you do NOT need to re-run cmake (configure). Just re-run the build step:

Incremental vs. clean rebuild

```
# Go to the build folder
cd ~/har_build

# Recompile only the changed files (cmake is smart about this)
cmake --build .

# To do a completely clean rebuild (delete everything and start over):
cd ~
rm -rf har_build
mkdir har_build
cd har_build
cmake "/mnt/f/.../OnDeviceTraining_HAR_FQT_RigL"
cmake --build .
```

11. Complete Project Workflow — Step by Step

Follow these steps in exact order from a fresh WSL terminal. Each step must finish without errors before you move to the next. Do NOT paste multiple commands at once.

Phase 1 — First-time setup (do only once)

Phase 1: Install tools

```
# Open WSL (search 'wsl' in Windows Start menu)
# You will see: matef@LAPTOP:~$

# 1a. Update package list
sudo apt update
# Wait for: 'Reading package lists... Done'

# 1b. Install build tools
sudo apt install -y cmake gcc g++ make
# Wait for: 'done' on the last line

# 1c. Verify
cmake --version
gcc --version
# Both should print version numbers without error
```

Phase 2 — Create the build folder (do once per clean build)

Phase 2: Configure

```
# 2a. Create a build folder in the Linux home directory
mkdir -p ~/har_build

# 2b. Go into it
cd ~/har_build

# 2c. Confirm you are in the right place
pwd
# Expected: /home/matef/har_build

# 2d. Configure the project (point cmake at the source code)
cmake "/mnt/f/technical material/Master thesis/claude/on_device
training/OnDeviceTraining_HAR_FQT_RigL"

# Expected final line:
# -- Build files have been written to: /home/matef/har_build
```

Phase 3 — Compile the project

Phase 3: Compile

```
# Still inside ~/har_build
cmake --build .

# You will see lines like:
# [ 2%] Building C object ...
# [ 12%] Building C object ...
# ...
# [100%] Built target har_fqt_rigl

# If you see errors, check Section 12 (Common Errors).
# If it says [100%] Built target har_fqt_rigl – you are done!

# Find the compiled executable:
ls ~/har_build/
# You should see: har_fqt_rigl (among other files)
```

Phase 4 — Prepare the dataset (do once)

Phase 4: Prepare dataset

```
# Navigate to the OnDeviceTraining root folder
cd "/mnt/f/technical material/Master thesis/claude/on_device training/OnDeviceTraining"

# Run the data preparation script
python3 examples/har_classifier/prepare_data.py

# This script downloads and converts the UCI-HAR dataset into .npy files:
# train_x.npy, train_y.npy
# val_x.npy, val_y.npy
# test_x.npy, test_y.npy

# Verify the files were created:
ls examples/har_classifier/data/
```

Phase 5 — Run the training

Phase 5: Train the model

```
# Run from the OnDeviceTraining root so the data paths resolve correctly
cd "/mnt/f/technical material/Master thesis/claude/on_device training/OnDeviceTraining"

# Start training
~/har_build/har_fqt_rigl

# The program will print:
# === Model Summary ===
# Epoch 1/20 ...
# Training Loss: 1.xxxx Train Acc: xx.x%
# ...
# === Sparsity Table (RigL) ===
# === FLOPs Report ===
# === Classification Report ===
# Confusion Matrix:

# To stop training at any time: press Ctrl+C
```

12. Common Errors and How to Fix Them

These are the errors most commonly encountered during this project. Each entry explains WHAT the error means, WHY it happens, and exactly HOW to fix it.

Error: '&&' is not a valid statement separator in this version.

Cause: You typed && in PowerShell, which does not support it as a command separator.

Fix: Replace && with ; or put each command on its own line:

```
mkdir build_pc
cd build_pc
```

Error: cmake: command not found

Cause: CMake has not been installed in WSL yet.

Fix: Run: `sudo apt update`

Then: `sudo apt install -y cmake gcc g++ make`

Error: CMake Error: Unknown argument -f

(or: cmake: invalid option -- 'f')

Cause: You tried to use `cmake -f` to specify a custom filename.

CMake does not have a -f flag. It always reads CMakeLists.txt.

Fix: Rename your file first:

```
Rename-Item CMakeLists_PC.txt CMakeLists.txt
```

Then run `cmake` without -f.

Error: CMake Error: The source directory does not appear to contain CMakeLists.txt.

Cause: `cmake` cannot find the source code. The path you gave is wrong, or CMakeLists.txt has not been renamed yet.

Fix: Check the path is correct (ls it first).

Make sure CMakeLists.txt exists (not CMakeLists_PC.txt).

Error: CMake Error: The current CMakeCache.txt directory is not the same as the directory CMakeCache.txt was written to. (or: not a CMake build directory)

Cause: You ran `cmake --build .` from the wrong folder. You must be inside the build folder (~\har_build), not the source folder.

Fix: `cd ~/har_build`

then try `cmake --build .` again.

Error: error: a label can only be part of a statement and a declaration is not a statement

Cause: You are using MinGW GCC 8.1 (the Windows GCC bundled with CodeBlocks). It is too old. The OnDeviceTraining framework requires GCC 10 or newer.

Fix: Switch to WSL. WSL comes with Ubuntu's GCC 12, which handles this correctly.

Error: rm: cannot remove 'build_pc': Permission denied
(or: The process cannot access the file because it is
being used by another process)

Cause: WSL has the folder locked open (open file handle). Windows cannot delete a folder that Linux has open.

Fix: In PowerShell/CMD:
wsl --shutdown
(wait 5 seconds)
Remove-Item -Recurse -Force build_pc
wsl

Error: mkdir: cannot create directory 'build_pc': File exists

Cause: The folder already exists. mkdir fails if the destination already exists.

Fix: In Linux: use mkdir -p (the -p flag suppresses this error):
mkdir -p build_pc
Or delete the existing folder first, then recreate it.

Error: Cannot open file 'train_x.npy' (or similar .npy file)

Cause: The dataset has not been prepared yet, or the program is being run from the wrong working directory.

Fix: Run prepare_data.py first:
python3 examples/har_classifier/prepare_data.py
Then run the executable FROM the OnDeviceTraining root folder.

13. Quick Reference Card

The tables below are a compact cheat sheet. Once you are comfortable with the explanations in earlier sections, you can use this as a lookup table.

Navigation

Task	Linux / WSL	PowerShell	CMD
Where am I?	pwd	pwd	cd
Go into folder	cd foldername	cd foldername	cd foldername
Go up one level	cd ..	cd ..	cd ..
Go to home	cd ~	cd \$HOME	cd %USERPROFILE%
Go to drive F:	cd /mnt/f	cd F:\	F:\
List files	ls or ls -la	ls or dir	dir

File and Folder Operations

Task	Linux / WSL	PowerShell	CMD
Create folder	mkdir name mkdir -p a/b/c	mkdir name	mkdir name
Delete file	rm file.txt	Remove-Item file.txt	del file.txt
Delete folder	rm -rf folder	Remove-Item -Recurse -Force folder	rmdir /s /q folder
Rename file	mv old new	Rename-Item old new	ren old new
Copy file	cp src dst	Copy-Item src dst	copy src dst
View file	cat file.txt	Get-Content file.txt	type file.txt
Chain (if ok)	cmd1 && cmd2	cmd1 ; cmd2	cmd1 && cmd2
Chain (always)	cmd1 ; cmd2	cmd1 ; cmd2	cmd1 & cmd2

WSL Commands (typed in PowerShell or CMD)

Command	What It Does
wsl	Open a WSL bash terminal
wsl --list	Show installed Linux distributions
wsl --shutdown	Stop all WSL instances (releases file locks)
wsl --version	Show WSL and kernel version
exit	Leave WSL (typed inside WSL)

apt Package Manager (typed in WSL)

Command	What It Does
sudo apt update	Refresh the package catalogue

sudo apt install -y cmake gcc g++ make	Install build tools
sudo apt remove package	Uninstall a package
apt search keyword	Search for a package

CMake Build Commands (typed in WSL, inside ~/har_build)

Command	What It Does
cmake "/mnt/ff/.../project"	Configure: read CMakeLists.txt and prepare build files
cmake --build .	Build: compile all C source files
cmake --build . -j4	Build using 4 CPU cores in parallel (faster)
cmake --build . --clean-first	Delete old compiled files and rebuild everything

Keyboard Shortcuts

Shortcut	What It Does	Works In
Ctrl+C	Stop the currently running program	All shells
Up Arrow	Recall the previous command	All shells
Down Arrow	Go forward in command history	All shells
Tab	Autocomplete file/folder names	All shells
Ctrl+L	Clear the screen	Linux/WSL, PowerShell
Ctrl+A	Jump to beginning of the line	Linux/WSL
Ctrl+E	Jump to end of the line	Linux/WSL